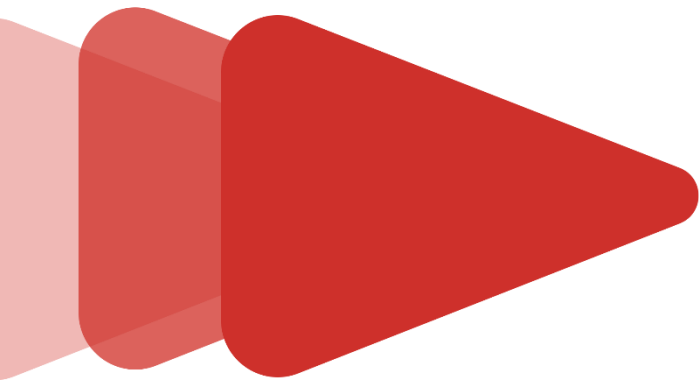




Arquitectura de la Solución Tecnológica Entrega y Recepción de Información Fiscal - Anexos

NAC-PTITARS25-00000027

***Gestión de Datos y Arquitectura Tecnológica
Departamento de Planificación TI y Gestión de Información -Datos
Dirección Nacional de Tecnología***

Diciembre 2025

Versión: 1.0.0

INFORMACIÓN DEL DOCUMENTO

Propósito:	El presente documento contiene las definiciones de arquitectura, estructura de componentes y su interacción para la Solución Tecnológica de Entrega y Recepción de Información Fiscal - Anexos.
Alcance:	Contempla las definiciones de arquitectura software, arquitectura de datos y arquitectura de infraestructura para el release 1 del proyecto que está desarrollando la aplicación de Anexos para la implementación integral del dominio funcional requerido para la administración, configuración, habilitación de anexos tributarios que permiten gestionar la estructura de anexos, la parametrización y catálogos operativos, la configuración de obligaciones, los canales de recepción, la habilitación de los anexos.
Audiencia:	Dirección Nacional de Tecnología

REVISIONES

Versión/Fecha	Autor	Descripción
1.0.0 / 19/12/2025	Santiago Simbaña	Emisión inicial para el release 1.

Firmas de Responsabilidad

Fecha: 19/12/2025	Dirección Nacional de Tecnología	VERSIÓN 1.0.0	PÁGINAS 28
CARGO		FIRMA	
APROBADO POR:	Ing. Paulina Correa A. JEFE NACIONAL DE DEPARTAMENTO DE PLANIFICACIÓN TI Y GESTIÓN DE INFORMACIÓN-DATOS		
REVISADO POR:	Ing. Edison Cayambe Ch. COORDINADOR NACIONAL DE GESTIÓN DE DATOS Y ARQUITECTURA TECNOLÓGICA		
COLABORA:	Ing. Sofia Sevilla A. EXPERTO NACIONAL DE DESARROLLO	 Firmado electrónicamente por: KAREN SOFIA SEVILLA AMAYA Validar únicamente con FirmaEC	
ELABORADO POR:	Ing. Santiago Simbaña H. EXPERTO NACIONAL DE ARQUITECTURA DE SOLUCIONES TECNOLÓGICAS	 Firmado electrónicamente por: SANTIAGO DAVID SIMBANA HAYO Validar únicamente con FirmaEC	

Contenido

1.	Introducción	5
2.	Arquitectura de la Solución Tecnológica – Entrega y Recepción de Información Fiscal - Anexos 6	
2.1.	Modelo de componentes externos y su interacción	6
2.1.1.	Solución Tecnológica	6
2.1.2.	Aplicación	6
2.2.	Arquitectura de software de la Solución Tecnológica – Entrega y Recepción de Información Fiscal - Anexos	8
2.2.1.	Definiciones de Arquitectura	8
2.2.2.	Modelo de componentes internos ambiente Intranet	10
2.3.	Arquitectura de datos de la Solución Tecnológica – Anexos	18
2.3.1.	Definiciones de Arquitectura	18
2.3.2.	Modelo de componentes internos	19
2.4.	Arquitectura de infraestructura de la Solución Tecnológica – Anexos	20
2.4.1.	Definiciones de Arquitectura	20
2.4.2.	Modelo de Despliegue Intranet - Anexos	21
3.	Glosario	23
3.1.	Acrónimos	23
3.2.	Definiciones	24
4.	Bibliografía	26
5.	Anexos	27
5.1.	Anexo A: Versión de tecnologías	27

1. Introducción

El Sistema de Anexos Tributarios constituye un componente esencial dentro de la gestión de información fiscal del Servicio de Rentas Internas. La plataforma vigente, construida hace más de dos décadas, presenta limitaciones que afectan su evolución funcional, su capacidad para manejar la complejidad actual de los anexos y su alineación con los estándares tecnológicos institucionales. Las necesidades operativas recientes —incluyendo la definición dinámica de estructuras, la administración de catálogos, la configuración de obligaciones, la parametrización de reglas y la habilitación interna de anexos para contribuyentes— evidencian la importancia de modernizar el modelo de construcción y gobierno de estas definiciones.

El Release 1 (Intranet) se concentra exclusivamente en la construcción y habilitación de la plantilla de anexos, estableciendo las bases para los procesos posteriores de recepción, validación y procesamiento que se implementarán en releases siguientes. En este alcance, el sistema debe permitir:

- Definir y versionar la estructura del anexo (secciones, casilleros y metadatos).
- Administrar catálogos funcionales propios del dominio.
- Configurar la relación entre anexos y obligaciones tributarias.
- Parametrizar reglas de negocio y condiciones asociadas a la visualización de campos en la plantilla.
- Determinar los canales institucionales por los cuales un anexo podrá ser recibido.
- Generar las integraciones con sistemas institucionales conforme las funcionalidades requeridas para: notificaciones, auditoría, catastro, obligaciones y consumo de tablas transversales

La presente arquitectura se diseña bajo principios institucionales de resiliencia, desacoplamiento y estandarización, empleando microservicios alineados aDDD, exposición controlada mediante API Management, autenticación centralizada con Keycloak. Asimismo, se adopta un modelo de despliegue en OpenShift on-premise, cumpliendo con los lineamientos de observabilidad, gobernanza y alta disponibilidad requeridos por el SRI.

Este documento consolida los componentes funcionales, las interacciones entre microservicios, los modelos de datos y las decisiones arquitectónicas que sustentan la implementación del Release 1 del Sistema de Anexos. El resultado es una plataforma modular, robusta y preparada para escalar en releases posteriores hacia la recepción masiva, validación operativa, procesamiento tributario y exposición para contribuyentes externos.

A continuación, se detalla la lista de la documentación de la solución tecnológica con sus respectivos repositorios:

DESCRIPCIÓN	ruta
Documentación de la arquitectura de la solución tecnológica.	Repositorio: ALFRESCO Repositorio> Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURAS, ESTÁNDARES Y GUÍAS> ARQUITECTURA TECNOLÓGICA DE LA SOLUCIÓN> 02 CADENA DE VALOR> 02 CONTROL AL CUMPLIMIENTO TRIBUTARIO> ENTREGA Y RECEPCIÓN DE INFORMACIÓN FISCAL> ANEXOS
Diagramas/modelos de la arquitectura de la solución tecnológica Anexos.	Repositorio: TEAMWORK SERVER Categoría: Arquitectura de la Solución Ubicación: Data/CADENA DE VALOR/Control al Cumplimiento Tributario/Entrega y Recepción de Información Fiscal/ Gestión Anexos
Requerimiento de Usuario de la aplicación Anexos.	Repositorio: ALFRESCO Repositorio Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURAS, ESTÁNDARES Y GUÍAS> ARQUITECTURA TECNOLÓGICA DE LA SOLUCIÓN> 02 CADENA DE VALOR> 02 CONTROL AL CUMPLIMIENTO TRIBUTARIO> ENTREGA Y RECEPCIÓN DE INFORMACIÓN FISCAL> ANEXOS> CONSTRUCCION> EDITABLES> ARS_25027_Anexos_vE1.0.0

2. Arquitectura de la Solución Tecnológica – Entrega y Recepción de Información Fiscal - Anexos

2.1. Modelo de componentes externos y su interacción

2.1.1. Solución Tecnológica

Enlace del modelo UML y descripción de los Componentes Externos de la Solución tecnológica Entrega y Recepción de información Fiscal – Anexos:

Url confluence:

<http://sriarqic01:1100/pages/viewpage.action?pageId=124617037> .

2.1.2. Aplicación

La aplicación tecnológica de Anexos se ha concebido como una aplicación que está disponible para su utilización desde los ambientes internet e intranet, cuyas principales interacciones pueden darse considerando las integraciones que se

muestran de manera gráfica de los componentes externos de la aplicación Entrega y Recepción de Información Fiscal.

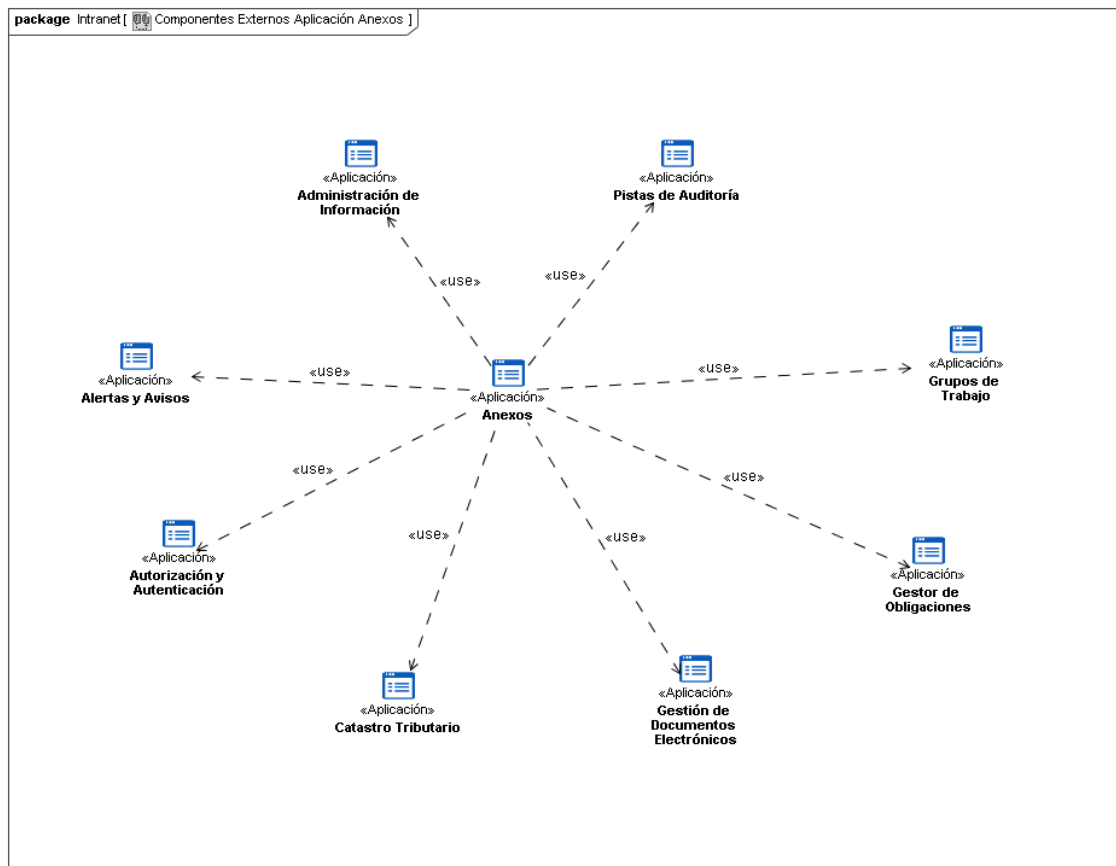


Figura 1: Modelo de Componentes Externos Entrega y Recepción de Información Fiscal - Anexos

Aplicación	Relación
Anexos	La aplicación Anexos constituye el núcleo funcional encargado de la gestión integral del ciclo de vida de los anexos tributarios en Internet e Intranet. Coordina la interacción con componentes transversales institucionales para obtener catálogos, validar contribuyentes, registrar obligaciones, auditar procesos y emitir notificaciones. Las integraciones se realizan mediante servicios REST gestionados en API Management, autenticados vía Keycloak y operados por los microservicios `sri-anexos-*.service`.
Administración de Información	Proporciona los catálogos institucionales utilizados por la solución Anexos para validar parámetros, tipos de impuestos, tasas, sanciones y metadatos transversales. La interacción se realiza por servicios REST que actúan como fuente única de datos maestros, garantizando consistencia y gobernanza institucional.
Autenticación y Autorización	Servicio centralizado basado en Keycloak/Red Hat SSO, utilizado para la validación de accesos y permisos mediante tokens OAuth2/OpenID. El sistema Anexos valida cada solicitud, obtiene roles autorizados y garantiza la seguridad en los flujos tanto internos como externos.
Alertas y Avisos	Permite emitir notificaciones automáticas hacia contribuyentes a través de correo y buzón electrónicos institucional. El sistema

	Anexos envía solicitudes estructuradas informando los resultados del procesamiento (aceptado, rechazado, con observaciones). La integración asegura trazabilidad y comunicación oportuna.
Catastro Contributivo	Repositorio oficial de datos del sujeto pasivo. El sistema Anexos consulta información del contribuyente (estado del RUC, razón social, tipo de contribuyente, obligaciones) para validar que el anexo sea presentado por el sujeto correcto y que cumpla con la normativa vigente.
Gestor de Obligaciones	Encargado de registrar y administrar el cumplimiento de obligaciones tributarias. El sistema Anexos comunica el resultado del procesamiento de los anexos para marcar obligaciones como cumplidas. El módulo GOB-Batch procesa estas marcas.
Grupos de trabajo	Herramienta que administra roles y configuraciones de usuarios internos del SRI. La solución Anexos se apoya en este componente para identificar perfiles autorizados para tareas operativas internas, auditorías o revisiones especializadas.
Pistas de Auditoria	Mecanismo transversal institucional para el registro de eventos críticos. El sistema Anexos registra eventos como autenticación, carga de archivos, validaciones, cambios de estado y resultados, cumpliendo normativas de seguridad y trazabilidad.

2.2. Arquitectura de software de la Solución Tecnológica – Entrega y Recepción de Información Fiscal - Anexos

2.2.1. Definiciones de Arquitectura

Definición: Utilizar un JDK compatible con OpenJDK, conforme a la versión establecida en el estándar institucional vigente. Para el presente release, se adopta GraalVM Community Edition como distribución del JDK para la ejecución de los servicios, garantizando el cumplimiento del estándar Java definido

Motivación: Se asegura la compatibilidad, estabilidad y soporte institucional en los entornos de ejecución, así como el cumplimiento de los lineamientos corporativos de seguridad, mantenimiento y arquitectura resiliente, establecidos en el Estándar de Backend – Arquitectura resiliente. Adicionalmente, la adopción de GraalVM permite optimizar el tiempo de arranque y el consumo de recursos en entornos contenerizados, manteniendo la portabilidad, gobernanza tecnológica y alineación con el estándar Java institucional.

Definición: Implementar Quarkus como framework backend para todos los servicios sri-anexos-* -service y adapters institucionales.

Motivación: Quarkus ofrece tiempos de arranque optimizados, menor huella de memoria y soporte nativo para entornos contenerizados, lo que mejora el rendimiento y escalabilidad de la arquitectura de microservicios.

Definición: Utilizar Vue.js como framework de frontend para el portal Intranet del Sistema de Anexos.

Motivación: Proporciona un modelo reactivo, alta productividad en el desarrollo y una experiencia de usuario moderna, alineada con los lineamientos institucionales de interfaces web.

Definición: Implementar servicios REST para la exposición e integración de funcionalidades del dominio Anexos.

Motivación: Facilita la interoperabilidad entre equipos y sistemas heterogéneos mediante HTTP estándar, simplificando consumo, prueba, documentación y mantenimiento.

Definición: Definir y documentar todas las APIs RESTful con OpenAPI antes de su implementación.

Motivación: Establece contratos claros desde etapas tempranas, permite generación automática de artefactos (mocking, SDKs, pruebas) y asegura gobernanza de APIs bajo el api management.

Definición: Utilizar RH-SSO/Keycloak como proveedor único de identidad para autenticación en los módulos de Intranet e Internet.

Motivación: Centraliza la gestión de sesiones, roles y permisos, reduce vulnerabilidades y garantiza cumplimiento de estándares institucionales de seguridad.

Definición: Contenerizar todos los componentes (frontend, backend, adapters, workers/event listeners) y desplegarlos en OpenShift.

Motivación: Aporta portabilidad, consistencia entre ambientes, autoscaling, monitoreo nativo y cumplimiento de la estrategia institucional de orquestación.

Definición: Todos los microservicios transaccionales del dominio Anexos deberán utilizar Oracle como motor de base de datos autorizado.

Motivación: Garantiza integridad, soporte institucional, alta disponibilidad y alineación con la arquitectura de datos institucional.

Definición: Uso obligatorio de Oracle como base de datos relacional para componentes transaccionales, y almacenamiento de estructuras JSON utilizando el tipo de dato nativo JSON de Oracle 21c o superior.

Motivación: Oracle a partir de la versión 21c ofrece soporte nativo optimizado para JSON como tipo de dato estructurado, lo que permite indexación eficiente, consultas con JSON_TABLE, validación semántica, mejor compresión y rendimiento superior frente al uso de tipos genéricos como CLOB. Además, mejora la integración con aplicaciones que manipulan objetos JSON (como microservicios en Quarkus).

Esta decisión refuerza la mantenibilidad, eficiencia y alineación con estándares modernos de almacenamiento híbrido (relacional + semiestructurado).

Definición: Toda comunicación con sistemas legados o institucionales se realizará a través de los microservicios definidos, sin acceso directo.

Motivación: Asegura desacoplamiento, uniformidad en la integración, simplifica monitoreo, auditoría y reduce riesgos de dependencia tecnológica.

Definición: Todas las APIs expuestas deberán pasar por el API Management.

Motivación: Permite control de tráfico, seguridad, versionamiento, límites, analítica y cumplimiento de políticas institucionales.

Definición: Aplicar Domain-Driven Design para la definición de bounded contexts, aggregates, entidades, VO y eventos en cada microservicio.

Motivación: Asegura claridad en límites funcionales, estabilidad arquitectónica y correcta evolución del dominio Anexos.

Definición: Todos los microservicios deberán integrar logging centralizado (EFK/ELK), métricas y trazabilidad distribuida.

Motivación: Permite diagnóstico, auditoría, supervisión activa y cumplimiento de normativas internas.

2.2.2. Modelo de componentes internos ambiente Intranet

El modelo de componentes internos del ambiente Intranet describe los elementos técnicos que conforman la solución del Sistema de Anexos – Release 1, sus responsabilidades, dependencias (directas y transitivas) y las tecnologías utilizadas para brindar los servicios de cada componente.

Este modelo se basa en una arquitectura de microservicios orientada a dominio (DDD), apoyada en eventos (EDA), expuesta mediante API Management corporativo y desplegada sobre infraestructura on-premise.

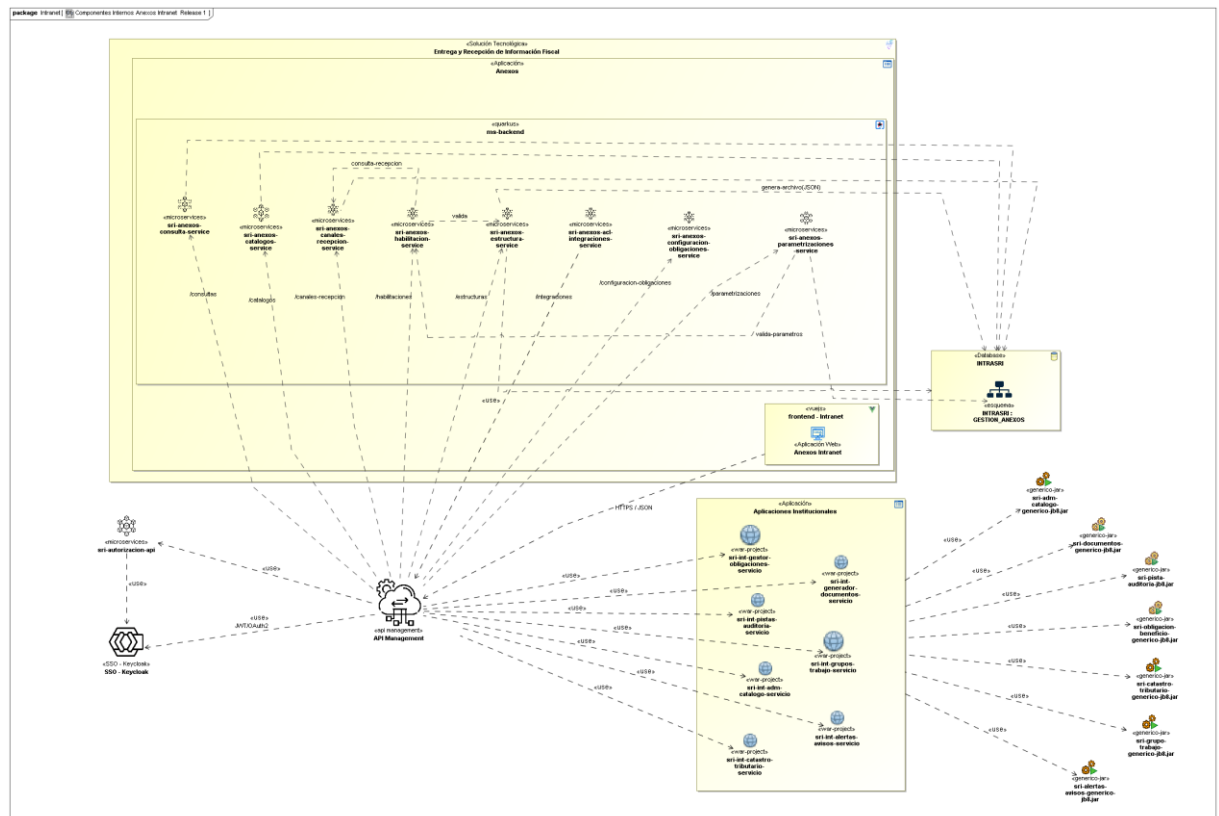


Figura 2: Modelo de Componentes Internos Intranet
Entrega y Recepción de Información Fiscal - Anexos

Artefacto/Microservicio	Descripción	Tecnología Utilizada
Anexos:		
sri-anexos-estructura-service	Descripción: Gestiona el modelo estructural del anexo: creación, actualización, versionado, estados de aprobación/publicación y previsualización. Tipo: Microservicio de dominio Bounded Context: Estructura de Anexos(definición de anexos, secciones, casilleros, versiones y estados de diseño/publicación) Agregados principales: Anexo, Sección, Casillero, VersionEstructura. Endpoints base: /api/v1/estructuras. Persistencia: Oracle INTRASRI, esquema GESTION_ANEXOS. Dependencias directas: API Management, Oracle INTRASRI	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit5 + RestAssured • JaCoCo • Sonar/PMD
sri-anexos-catalogos-service	Descripción: Administra catálogos parametrizables usados por otros microservicios del dominio Anexos. Tipo: Microservicio de dominio Bounded Context: Catálogos de Anexos.	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A

	Agregados principales: CatalogoAnexo, ValorCatalogoAnexo. Endpoints base: /api/v1/catalogos. Persistencia: Oracle INTRASRI, esquema GESTION_ANEXOS Dependencias directas: API Management, Oracle INTRASRI. Dependencias transitivas: Servicios de Anexos que leen estos catálogos (estructuras, parametrizaciones, habilitación).	• Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit5 + RestAssured • JaCoCo • Sonar/PMD
sri-anexos-parametrizacion-service	Descripción: Gestiona reglas de negocio declarativas que se aplicarán sobre las estructuras de anexos (solo definición y persistencia en R1). Tipo: Microservicio de dominio Bounded Context: Parametrización de Anexos (reglas, visibilidad, mapeos campo-fuente). Agregados principales: ParametrizacionAnexo, ReglaValidacion, ReglaCalculo, CondicionVisibilidad, MapeoCampoFuente. Endpoints base: /api/v1/parametrizaciones. Persistencia: Oracle INTRASRI, esquema GESTION_ANEXOS. Dependencias directas: API Management, Oracle INTRASRI. Dependencias transitivas: sri-anexos-habilitacion-service.	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit5 + RestAssured • JaCoCo • Sonar/PMD
sri-anexos-configuracion-obligaciones-service	Descripción: Define qué anexos aplican a qué obligaciones, segmentos y vigencias; insumo clave para habilitación de anexos. Tipo: Microservicio de dominio Bounded Context: Configuración Anexo-Obligación. Agregados principales: ConfiguracionObligacionAnexo, SegmentoContribuyente, VigenciaConfiguracion. Endpoints base: /api/v1/configuracion-obligaciones. Persistencia: Oracle INTRASRI, esquema GESTION_ANEXOS. Dependencias directas: API Management, Oracle INTRASRI, sri-gestor-obligaciones-service. Dependencias transitivas: sri-anexos-habilitacion-service.	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit5 + RestAssured • JaCoCo • Sonar/PMD
sri-anexos-canales-recepcion-service	Descripción: Define qué canales de recepción están habilitados para cada anexo (formulario web, WS, archivo, FTP, etc.), solo a nivel de configuración en este release. Tipo: Microservicio de dominio Bounded Context: Canales de Recepción (Configuración).	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive

	Agregados principales: ConfiguracionCanalesAnexo, CanalRecepcion. Endpoints base: /api/v1/canales-recepcion. Persistencia: Oracle INTRASRI, esquema GESTION_ANEXOS. Dependencias directas: API Management, Oracle INTRASRI. Dependencias transitivas: sri-anexos-habilitacion-service.	• OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit5 + RestAssured • JaCoCo • Sonar/PMD
sri-anexos-habilitacion-service	Descripción: Orquesta el proceso de habilitación del anexo, pasando por los diferentes estado y aprobaciones correspondientes de los usuarios asignados como (analistas, supervisores y/o aprobadores) Tipo: Microservicio de dominio Bounded Context: Habilitación de Anexos. Agregados principales: HabilitacionAnexo, EstadoHabilitacion, ObservacionHabilitacion. Endpoints base: /api/v1/habilitaciones. Persistencia: Oracle INTRASRI, esquema GESTION_ANEXOS. Dependencias directas: API Management, Oracle INTRASRI, sri-anexos-estructura-service, sri-anexos-parametrizacion-service, sri-anexos-configuracion-obligaciones-service, sri-anexos-canales-recepcion-service. Dependencias transitivas: sri-anexos-acl-integraciones-service.	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit5 + RestAssured • JaCoCo • Sonar/PMD
sri-anexos-consultas-service	Descripción: Expone consultas y vistas consolidadas sobre configuración y habilitaciones de anexos para uso operativo de usuarios internos. Tipo: Microservicio de lectura / consultas Bounded Context: Consultas Operativas de Anexos (Intranet). Agregados principales: Read models/proyecciones: ResumenHabilitacion, DetalleConfiguracion, etc. Endpoints base: /api/v1/consultas. Persistencia: Oracle INTRASRI (ANEXOS) y, a futuro, Oracle INTERSRI (proyecciones para Internet). Dependencias directas: API Management, Oracle INTRASRI; en R2 puede consultar Oracle INTERSRI. Dependencias transitivas: Depende de los datos generados.	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit5 + RestAssured • JaCoCo • Sonar/PMD
sri-anexos-acl-integraciones-service	Descripción: Consume eventos de dominio y orquesta integraciones hacia servicios institucionales (sri-int-*) a través del API Management. Tipo: Microservicio de ACL / Integración Bounded Context: ACL de Integraciones del dominio Anexos.	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive

	Agregados principales: TareaIntegracion, TrazabilidadIntegracion. Endpoints base: /api/v1/integraciones. Persistencia: Oracle INTRASRI (tarefas de integración y trazas). Dependencias directas: API Management, Oracle INTRASRI, servicios sri-int-*. Dependencias transitivas: Sistemas legados JEE (ADM, Catastro, Pista auditoria, generador documentos, grupos de trabajo, alertas y avisos) a través de los adapters sri-int-*.	• OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit5 + RestAssured • JaCoCo • Sonar/PMD
Frontend Intranet		
Aplicativo Web Anexos Intranet	Aplicación web usada por usuarios internos para diseñar anexos, configurar parametrizaciones, obligaciones, canales y gestionar habilitaciones y consultas. Consume APIs REST del dominio Anexos.	• Vue versión: Anexo A. • Node versión: Anexo A. • NPM versión: Anexo A. • Vue CLI versión: Anexo A. • TypeScript versión: Anexo A. • Vite versión: Anexo A. • PrimeVue versión: Anexo A. • Primeicons versión: Anexo A. • Bootstrap versión: Anexo A. • Pinia versión: Anexo A. • Axios • Keycloak-js versión: Anexo A. • Formulajs: Anexo A. • vuelidate/core: Anexo A. • vuelidate/validators: Anexo A. • jexl: Anexo A. • uuid: Anexo A. • vuedraggable: Anexo A.
API Management		
API Management	Puerta de entrada única a todas las APIs del dominio Anexos e integraciones; aplica políticas de seguridad, rate limiting, trazabilidad y versionado. Aísla al dominio de consumidores y de detalles de exposición.	
Autenticación / SSO		

SSO / Keycloak	Proveedor de identidad corporativo; emite tokens OAuth2/OIDC usados por API Management y microservicios para validar identidad y roles.	SSO version: Anexo A .
sri-autorizacion-api	Descripción: Gestiona permisos y roles específicos del dominio Anexos, complementando la autenticación global de Keycloak. Tipo: Microservicio de dominio (seguridad de negocio) Bounded Context: Seguridad del dominio Anexos (autorización funcional). Agregados principales: Rol, Permisos, AsignacionPermisosUsuario. Endpoints base: /api/v1/autorizacion. Persistencia: Oracle INTRASRI. Dependencias directas: API Management, Oracle INTRASRI, Keycloak. Dependencias transitivas: Resto de microservicios sri-anexos-* consultan o usan su librería para autorización fine-grained.	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit5 + RestAssured • JaCoCo • Sonar/PMD
Base de datos		
INTRASRI	La aplicación de anexos utiliza la base de datos INTRASRI para guardar/modificar/consultar toda la información transaccional de esta aplicación en intranet, para lo cual utiliza el esquema denominado GESTION_ANEXOS .	• Oracle Database version: Anexo A.
Aplicaciones institucionales		
sri-int-gestor-obligaciones-servicio	Descripción: Expone servicios del Gestor de Obligaciones: consulta de obligaciones tributarias (periodicidad, vencimiento, estado) y registro de obligación cumplida cuando se presenta el anexo. Tipo: Servicio Web REST de integración / fachada Gestor de Obligaciones.	• Java JDK versión: Anexo A. • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jakarta Restful Web Services: Anexo A
sri-int-generador-documentos-servicio	Descripción: Expone los servicios del Generador de Documentos para elaborar el documento de resultado de carga/habilitación de anexos (talón resumen, constancias, reportes PDF) según plantillas institucionales. Tipo: Servicio Web REST de integración / generación de documentos.	• Java JDK versión: Anexo A. • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jakarta Restful Web Services: Anexo A

sri-int-pistas-auditoria-servicio	Descripción: Expone la solución de Pistas de Auditoría para registrar accesos y acciones críticas sobre anexos (consultas, descargas de talón resumen y archivos cargados) y permitir su consulta posterior. Tipo: Servicio Web REST de integración / Auditoría.	• Java JDK versión: Anexo A. • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jakarta Restful Web Services: Anexo A
sri-int-grupos-trabajo-servicio	Descripción: Expone información de grupos de trabajo/unidades responsables (direcciones nacionales, zonales, equipos de asistencia) usada para asignar responsables de revisión, aprobación y notificación de anexos. Tipo: Servicio Web REST de integración / organización interna.	• Java JDK versión: Anexo A. • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jakarta Restful Web Services: Anexo A
sri-int-adm-catalogo-servicio	Descripción: Provee acceso a las tablas transversales de Administración de Información (ADM Año, Mes, Países, Provincias, Cantones, Parroquias, Obligaciones, Identificaciones, etc.), con navegación jerárquica según el nivel requerido. Tipo: Servicio Web REST de integración / catálogos transversales.	• Java JDK versión: Anexo A. • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jakarta Restful Web Services: Anexo A
sri-int-alertas-avisos-servicio	Descripción: Provee operaciones para enviar notificaciones internas (correos, avisos) usando la solución institucional de Alertas y Avisos: publicación de anexo, solicitud de revisión, rechazos, etc. Tipo: Servicio Web REST de integración / mensajería de Alertas y Avisos.	• Java JDK versión: Anexo A. • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jakarta Restful Web Services: Anexo A
sri-int-catastro-tributario-servicio	Descripción: Expone servicios de consulta de información del contribuyente (RUC, razón social, medios de contacto, atributos adicionales) a partir del sistema legado de Catastro Tributario.	• Java JDK versión: Anexo A. • Jakarta Enterprise Beans: Anexo A

	Tipo: Servicio Web REST de integración / fachada Catastro Tributario.	• Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jakarta Restful Web Services: Anexo A
Genéricos		
sri-adm-catalogo-generico-jb8.jar	Componente requerido por la aplicación para la obtención de datos de catálogos transversales que corresponden a la administración del SRI, tales como: usuarios, estructura organizacional.	• JDK versión: Anexo A • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jackson Anexo A
sri-catastro-tributario-generico-jb8.jar	Componente requerido por la aplicación para obtener los datos del registro único de contribuyentes esto es: número ruc, razón social, tipo y clase de contribuyente, jurisdicción, representante legal, fecha de inicio y reinicio de actividades, fecha inicio y fin de obligado a llevar contabilidad, fallecido y si es fantasma o no.	• JDK versión: Anexo A • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jackson Anexo A
sri-obligacion-beneficio-generico-jb8.jar	Componente requerido por la aplicación para obtener información de consulta de obligaciones tributarias y registro de obligación.	• JDK versión: Anexo A • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jackson Anexo A
sri-grupo-trabajo-generico-jb8.jar	Componente requerido por la aplicación para la obtener los datos de los servidores con su respectiva información de roles institucionales.	• JDK versión: Anexo A • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jackson Anexo A
sri-documentos-generico-jb8.jar	Componente requerido por la aplicación para obtener el documento firmado de la contestación realizada a un trámite.	• JDK versión: Anexo A • Jakarta Enterprise Beans: Anexo A

		• Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jackson Anexo A
sri-alertas-aviso-generico-jb8.jar	Componente requerido por la aplicación para el envío de notificaciones a los contribuyentes y usuario internos.	• JDK versión: Anexo A • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jackson Anexo A
sri-pista-auditoria-jb8.jar	Componente requerido por la aplicación para el registro de pistas de auditoría de consultas transaccionales y funcionalidad crítica a través del uso de anotaciones.	• JDK versión: Anexo A • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jackson Anexo A

2.3. Arquitectura de datos de la Solución Tecnológica – Anexos

2.3.1. Definiciones de Arquitectura

Definición: Utilizar la base de datos Oracle para el almacenamiento de los datos generados por la aplicación.

Motivación: Oracle es un RDBMS para base de datos relacionales usado en la institución.

Definición: La base de datos a ser utilizada por la aplicación de Anexos en Intranet es INTRASRI.

Motivación: Utilizar la base de datos establecida para las aplicaciones de intranet con la arquitectura vigente.

Definición: Repositorio transaccional primario en el entorno Intranet del SRI. Almacena toda la información operativa del ciclo de vida de los anexos: creación, edición, validación, auditoría, estados y control de usuarios. Toda escritura ocurre exclusivamente en esta base.

Motivación: Garantiza la integridad referencial, seguridad, transaccionalidad ACID y trazabilidad de datos críticos, en un entorno controlado y no expuesto directamente a internet. Alineado con el principio de “fuente de verdad” operativa.

Definición: Base de datos de solo lectura en el entorno Internet, sincronizada desde INTRASRI. Se utiliza para exponer datos procesados de los anexos a contribuyentes a través del portal SRI en Línea. Prohibido realizar escrituras directas.

Motivación: Separa ambientes operativos y públicos, evitando comprometer el entorno transaccional. Aumenta la seguridad, mejora el rendimiento de consulta y reduce la exposición directa de datos internos.

Definición: El esquema a ser utilizado por la aplicación es GESTION_ANEXOS.

Motivación: Dar cumplimiento a los estándares institucionales en cuanto al nombrado de objetos de bases de datos.

2.3.2. Modelo de componentes internos

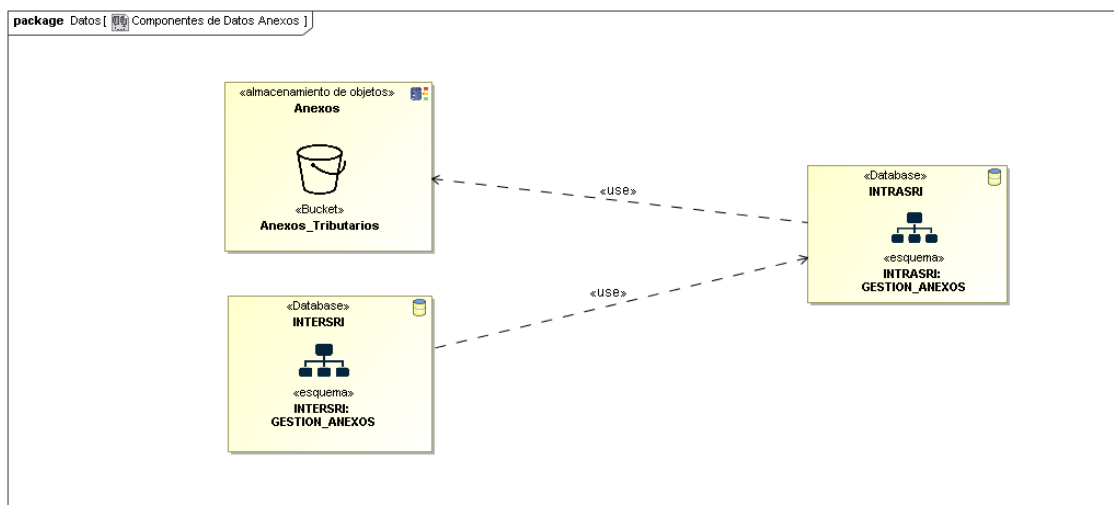


Figura 4: Modelo de Componentes Internos de Datos - nivel de diseño Intranet
Entrega y Recepción de Información Fiscal - Anexos

Diagrama de datos para las aplicaciones: Aplicación y su interacción con la(s) base de datos y esquema(s)

Componente	Descripción	Tecnología Utilizada
INTRASRI		
INTRASRI: GESTION_ANEXOS	Esquema de la aplicación Anexos donde se gestiona toda la información transaccional de esta aplicación en ambiente intranet.	Esquema de base de datos Oracle versión Anexo A .
INTERISRI		
INTERISRI: GESTION_ANEXOS	Esquema de la aplicación Anexos donde se gestiona toda la información transaccional de esta aplicación en	Esquema de base de datos Oracle versión

	ambiente internet. Esta base de datos corresponde al entorno Internet del SRI y actúa como un repositorio de solo lectura sincronizado a partir de los datos almacenados en INTRASRI. Su función principal es exponer de forma segura la información de anexos ya procesados a los contribuyentes a través de SRI en Línea u otras interfaces públicas.	Anexo A.
ALMACENAMIENTO DE OBJETOS		
Bucket Anexos_Tributarios	Fuente de datos semiestructurada donde se almacenan los archivos asociados a los anexos tributarios (principalmente documentos JSON/XML). Este componente de Bucket permitirá recibir y almacenar temporalmente la información de los anexos de gran volumen de datos a ser procesados y posterior eliminación	- Almacenamiento de objetos compatible S3/ODF (cluster on-premise). - Objetos en formato JSON/XML

2.4. Arquitectura de infraestructura de la Solución Tecnológica – Anexos

2.4.1. Definiciones de Arquitectura

Definición: La estrategia de alta disponibilidad y escalamiento horizontal para los componentes de la presente solución se implementará mediante la orquestación de contenedores en Kubernetes. El balanceo de carga se realizará a través de Kubernetes Services (tipo ClusterIP, NodePort o LoadBalancer) y será gestionado dinámicamente mediante controladores de replicación como Deployments o StatefulSets, los cuales aseguran la disponibilidad de múltiples réplicas en modo stateless o standalone.

Motivación: Establecer el mecanismo basado en contenedores y orquestación con Kubernetes que permita a la solución escalar horizontalmente y garantizar alta disponibilidad de sus componentes. Este enfoque busca asegurar que, ante un aumento en la demanda o en caso de fallos, la plataforma pueda adaptarse dinámicamente mediante la creación y balanceo automático de réplicas, sin necesidad de intervención manual o dependencias en servidores físicos específicos.

Definición: La aplicación debe ser desplegada utilizando OpenShift como plataforma de orquestación de contenedores, permitiendo una gestión eficiente, escalabilidad automática y alta disponibilidad del servicio.

Motivación: El uso de OpenShift permite mejorar la resiliencia y disponibilidad de la aplicación, facilitando el despliegue continuo y la recuperación ante fallos. Dado que se trata de un servicio crítico, es fundamental garantizar su operatividad constante y su capacidad de escalar según la demanda de los contribuyentes.

2.4.2. Modelo de Despliegue Intranet - Anexos

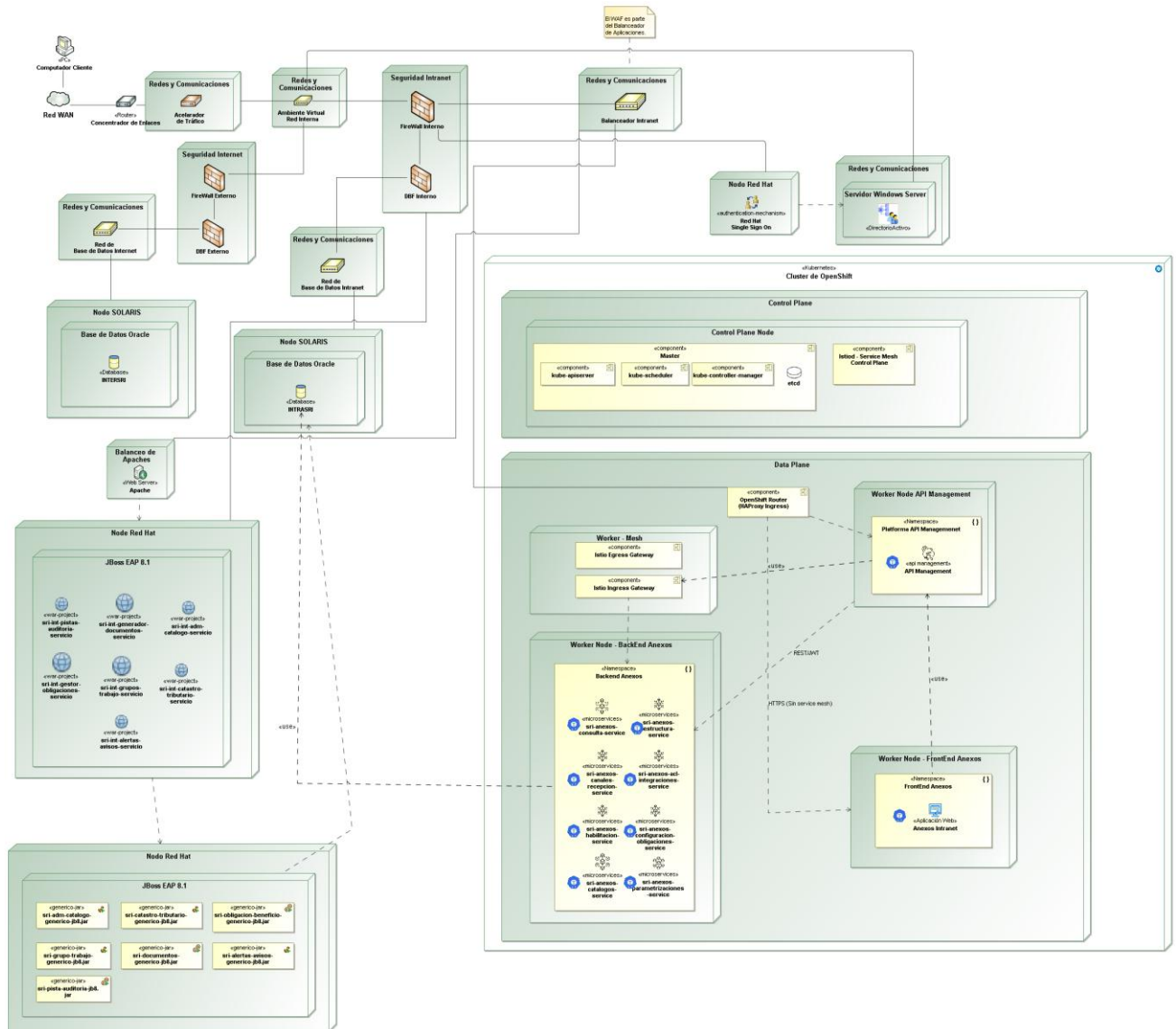


Figura 6: Modelo de Despliegue de la Solución Tecnológica Entrega y Recepción de Información Fiscal – Anexos

Nota:

- Los componentes de Seguridad Internet y Seguridad Intranet se encuentran descritos en la arquitectura general “Arquitectura Tecnológica Seguridad Informática”.
- Los componentes de Redes y Comunicaciones se encuentran descritos en la arquitectura general “Arquitectura Tecnológica Redes y Comunicaciones”.

Equipo	Características
Computador Cliente	Navegador: ver Anexo A
Red Hat Single Sign On Internet	Componente que se utiliza para la autenticación del aplicativo. Red Hat Single Sign-On.
Cluster OpenShift	Plataforma de contenedores corporativa. Aloja todos los pods de Anexos, integraciones, API Mgmt, mesh, etc. Cluster OCP instalado y operativo (masters + workers).

Equipo	Características
Master Node	Nodos que ejecutan API Server, Scheduler, Controller Manager y etcd. También alojan istiod (Service Mesh control plane). No ejecutan pods de negocio.
Control Plane	Conjunto de nodos y procesos que gobiernan el clúster Kubernetes/OpenShift: exponen la API del clúster, deciden dónde se ejecuta cada pod, mantienen el estado deseado y almacenan la configuración crítica (etcd). En él también reside el control plane del Service Mesh (istiod), que distribuye configuración y certificados mTLS a los sidecars. En el Control Plane no se ejecutan cargas de negocio; solo componentes de plataforma. Su indisponibilidad afecta a todo el clúster (no se pueden crear/actualizar pods, aunque los existentes sigan corriendo).
Kube Controller Manager	Kubernetes opera con un concepto de bucle de control, que concilia continuamente el estado real del clúster con el deseado. El administrador de controladores supervisa estos controladores para garantizar que las cargas de trabajo cumplan con los requisitos especificados.
Kube API Server	El servidor API de Kubernetes es el punto de entrada para interactuar con el clúster. Los usuarios se comunican con el clúster a través de esta API, que coordina las acciones de otros componentes según sea necesario.
Kube Scheduler	La función del programador es asignar cargas de trabajo (pods) a los nodos en función de los recursos disponibles, como CPU y memoria, para optimizar el rendimiento de las aplicaciones en todo el clúster.
etcd	Kubernetes utiliza etcd, un almacén de clave-valor de alta disponibilidad, como base de datos para almacenar toda la información relacionada con el clúster. Esto garantiza la consistencia de los datos de recursos en todos los nodos.
Istio / OpenShift Service Mesh – Control Plane	Componente que gestiona configuración del mesh: certificados, políticas mTLS, VirtualServices, DestinationRules.
Data Plane	Conjunto de nodos worker donde realmente se ejecutan los contenedores de negocio: microservicios de Anexos, microservicios de integración, frontend, API Management, gateways de mesh. El Data Plane procesa el tráfico de usuario y de sistema; aquí viven los sidecars Envoy del Service Mesh. El diseño separa el Data Plane en pools de workers por rol (Backend, Integraciones, Frontend, Infra, Mesh) para aislar cargas, facilitar el escalado y limitar el “blast radius” ante fallos.
Router OpenShift / Ingress Controller	Termina TLS de las Rutas, expone Frontend, API Mgmt, Keycloak hacia la red Intranet.
pods	El pod es la unidad lógica más pequeña de Kubernetes. Un pod contiene uno o más contenedores que se ejecutan en un nodo de trabajo.
Worker Node - BackEnd	Worker node encargado de contener a todos los componentes de la aplicación donde se ejecutan los microservicios de dominio Anexos, con sidecar Envoy (mesh).

Equipo	Características
Microservicios Backend Anexos (Quarkus)	Servicios REST de dominio Anexos, responsivos y reactivos (Quarkus, Hibernate Reactive, Oracle client). Imágenes Docker por servicio (sri-anexos-*.service), Deployments, Services ClusterIP, configuración de health checks.
Worker Node - FrontEnd	Worker Node encargado de gestionar los pods requeridos donde se despliega el frontend Intranet (Vue.js) empaquetado como contenedor.
Frontend Intranet Anexos (Vue.js)	Aplicación web que consume APIs de Anexos a través de API Mgmt y SSO Keycloak. Imagen Docker/Nginx con artefacto estático generado por Vue; Deployment + Service + Route.
Worker Node - API Management	Worker Node encargado de gestionar los pods deseados del API Management, aislados del backend.
Worker Mesh	Worker Node, donde residen Istio Ingress Gateway y Istio Egress Gateway, que manejan todo el tráfico que entra/sale del mesh.
Worker Node – Integraciones	Worker Node encargado de gestionar los pods de los microservicios de Integraciones hacia los sistemas legados, cada uno con sidecar Envoy.
Microservicios de Integración (sri-int-*)	Adaptadores REST/ SOAP que encapsulan el acceso a sistemas legados JEE; solo consumidos por sri-anexos-acl-integraciones-service. Imágenes Quarkus por cada sri-int-*, Deployments, Services, configuración de endpoints de legados, serialización XML/JSON según corresponda.
INTRASRI	Oracle Database versión Anexo A , instalado en un sistema operativo Solaris. Mediante el firewall de base de datos (DBF) son auditadas las acciones ejecutadas desde la aplicación Anexos y desde cualquier cliente contra esta base de datos.
JBoss EAP	Servidor de Aplicaciones Red Hat JBoss EAP y JDK de acuerdo con lo descrito en Anexo A, instalado en un Red Hat Linux, este servidor de aplicaciones destinado para desplegar los servicios web tipo REST de las integraciones y convivencias.

3. Glosario

Organizar glosario en orden alfabético

3.1. Acrónimos

BI	Business Intelligence
API	Application Programming Interface
Rest	Representational State Transfer
RH-SSO	Red Hat Single Sign-On
SRI	Servicio de Rentas Internas
EDA	Event-Driven Architecture
JDK	Java Development Kit
JVM	Java Virtual Machine (mencionado implícitamente por contexto JDK)
BI_DWH	Business Intelligence – Data Warehouse
etcd	Consistent and Highly-Available Key-Value Store for Kubernetes

API Mgmt	API Management (3scale o gateway institucional)
DDD	Domain-Driven Design
BFF	Backend for Frontend (si se menciona en otros diagramas)
OIDC	OpenID Connect
OAuth2	Protocolo de autorización
JSON-B	Java API for JSON Binding
AMQ	Red Hat AMQ / Apache Kafka (según uso institucional)
EAP	Enterprise Application Platform (JBoss/EAP)
S3/ODF	Objeto distribuido institucional (Object Data Fabric)
EFK/ELK	Stack de logging (Elasticsearch, Fluentd/Fluentbit, Kibana)
CI/CD	Integración y despliegue continuo
BPM/BPMS	Business Process Management Suite
DAL	Data Access Layer (implícito en integraciones)
ACL	Anti-Corruption Layer
R1	Release 1
SPA	Single Page Application
DNS	Domain Name System (Mesh/Services)
ConfigMap	Configuración externa en Kubernetes
K8s	Kubernetes

3.2. Definiciones

REST.- Arquitectura de servicios web que utiliza verbos HTTP estándar, lo que simplifica su implementación y comprensión. Facilita la interacción entre diferentes plataformas y sistemas.

Quarkus.- Framework de BackEnd optimizado para tiempos de arranque rápidos y bajo consumo de recursos, ideal para microservicios en Java.

Vue.js.- Framework de FrontEnd que permite construir interfaces reactivas y fáciles de desarrollar, mejorando la experiencia del usuario y la productividad del desarrollador.

Microservicios.- Arquitectura de desarrollo que descompone la aplicación en servicios pequeños y autónomos, cada uno enfocado en una capacidad de negocio específica. Mejora la modularidad, escalabilidad y el despliegue independiente.

Service Mesh.- es una capa de infraestructura que maneja la comunicación entre microservicios Descubrimiento de servicios, proporcionando las siguientes funciones:

- Balanceo de carga.
- Resiliencia
- Seguridad
- Monitoreo y trazabilidad

Event-Driven Architecture (EDA).- Modelo arquitectónico que permite a los componentes del sistema reaccionar de forma asincrónica a eventos. Mejora el desacoplamiento, la escalabilidad horizontal y la resiliencia.

OpenAPI.- Especificación para la definición y documentación de APIs RESTful y SOAP. Permite estandarización e interoperabilidad.

Kubernetes.- Plataforma de orquestación de contenedores que permite desplegar, escalar y gestionar aplicaciones en contenedores de forma automatizada y resiliente.

Pod.- Unidad lógica más pequeña en Kubernetes, que contiene uno o más contenedores ejecutándose en un nodo.

Node (Nodo).- Servidor físico o virtual dentro de un clúster de Kubernetes donde se ejecutan los pods.

Control Plane.- Componente de Kubernetes que administra el clúster y toma decisiones globales, como la programación de pods y la detección de eventos.

Data Plane.- Donde se ejecutan las aplicaciones del usuario, es decir, las cargas de trabajo en los nodos de trabajo.

Kubelet.- Agente que se ejecuta en cada nodo de Kubernetes y gestiona el ciclo de vida de los contenedores.

kube-proxy.- Componente que gestiona el tráfico de red entre pods en diferentes nodos dentro del clúster.

Etcd.- Base de datos clave-valor distribuida utilizada por Kubernetes para almacenar la configuración del clúster de forma consistente.

API Server.- Componente central de Kubernetes que expone la API y sirve de punto de entrada para la gestión del clúster.

Kube Scheduler.- Encargado de asignar pods a nodos disponibles según recursos y políticas definidas.

Streaming de Eventos.- Sistema para la comunicación en tiempo real, utilizando arquitectura basada en eventos para procesar mensajes de forma eficiente y no bloqueante.

Almacenamiento de Objetos.- Modelo para almacenar datos como objetos individuales (ejemplo: almacenamiento de archivos XML).

Data Fabric.- Capa de conectividad y orquestación de datos que permite acceder, integrar y gobernar datos de forma centralizada, habilitando análisis y toma de decisiones confiables.

Bucket.- Contenedor lógico en un sistema de almacenamiento de objetos, con políticas de retención definidas (por ejemplo, buckets para datos frecuentes, poco frecuentes o definitivos).

Namespace.- Contenedor lógico para organizar y aislar recursos en almacenamiento de objetos, compatible con protocolos como Amazon S3.

ACID.- es un conjunto de propiedades que garantizan la fiabilidad en las transacciones dentro de sistemas de bases de datos. El término proviene de:

- Atomicidad (Atomicity): Cada transacción se ejecuta como una unidad indivisible: o se completa totalmente o no se realiza en absoluto. Si ocurre un fallo, se revierte todo lo que se haya hecho.
- Consistencia (Consistency): Después de una transacción, la base de datos debe pasar de un estado válido a otro estado válido, respetando todas las reglas, restricciones y relaciones definidas.
- Aislamiento (Isolation): Las transacciones concurrentes no deben interferir entre sí. Cada transacción debe ejecutarse como si fuera la única en el sistema, evitando efectos colaterales.
- Durabilidad (Durability): Una vez que una transacción se confirma (commit), sus cambios persisten incluso ante fallos del sistema, gracias a mecanismos como registros (logs) y respaldos.

4. Bibliografía

The New Stack. (n.d.). How Redis simplifies microservices design patterns. The New Stack. <https://thenewstack.io/how-redis-simplifies-microservices-design-patterns/>

Fowler, M. (2011, 14 de julio). CQRS. Martin Fowler's Bliki. <https://martinfowler.com/bliki/CQRS.html>

Newman, C. (n.d.). Saga pattern. Microservices.io. Retrieved February 21, 2025, from <https://microservices.io/patterns/data/saga.html>

XenonStack. (2024, December 18). Microservices Architecture and Design Patterns. Retrieved from <https://www.xenonstack.com/insights/microservices>

Matei, Z., & Xin, R. (2022). Apache Spark: Unified Engine for Big Data Processing. Apache Software Foundation. <https://spark.apache.org/>

Red Hat. (2023). Red Hat OpenShift Documentation. <https://docs.openshift.com>

Oracle Corporation. (2023). Oracle Database JSON Developer's Guide. <https://docs.oracle.com/en/database/oracle/oracle-database/21/adjsn/>

Red Hat. (2023). 3scale API Management Documentation.
https://access.redhat.com/documentation/en-us/red_hat_3scale/

5. Anexos

5.1. Anexo A: Versión de tecnologías

Componente	Versión
BackEnd	
JDK	Red Hat OpenJDK 21.0.8. con su última actualización y/o parche liberado.
Quarkus	3.29
Lombok	1.18.42
MapStruct	1.6.3.redhat-00001
FrontEnd	
Vue	3.5.12
Node	22.9.0
NPM	10.8.3
Vue Cli	5.0.8
TypeScript	5.9.3
Vite	7.2.2
PrimeVue	4.3.7
Primeicons	7.0.0
BootStrap	5.3.7
Pinia	3.0.4
Axios	1.13.2
Keycloak-js	26.2.0
formulajs	4.5.5
vuelidate/core	2.0.3
vuelidate/validators	2.0.4
jexl	2.3.0
uuid	13.0.0
vuedraggable	4.1.0
Especificación Jakarta EE 10	
JDK	Red Hat OpenJDK 21.0.8. con su última actualización y/o parche liberado.
JBoss	EAP 8.1.0. con su última actualización y/o parche liberado.
Jakarta Persistence	3.1
Jakarta Contexts and Dependency Injection	4.0
Jakarta Servlet	6.0
Jakarta Enterprise Beans	4.0
Jakarta Bean	3.0

Validation	
Jackson	2.17
Jakarta Interceptors	2.1
Jakarta Transactions	2.0
Jakarta Restful Web Services	JAX-RS 3.1
Base de datos	
INTRASRI	Oracle 12.2.0.1c Enterprise Edition
STAR	Oracle 19.20c Enterprise Edition
Red Hat Single Sign On	7.1.4.GA
Linux	Red Hat 10.x con su última actualización y/o parche liberado
Navegadores	• Mozilla Firefox 146.0 o Superior • Google Chrome 110 o Superior • Microsoft Edge 143.0 o Superior